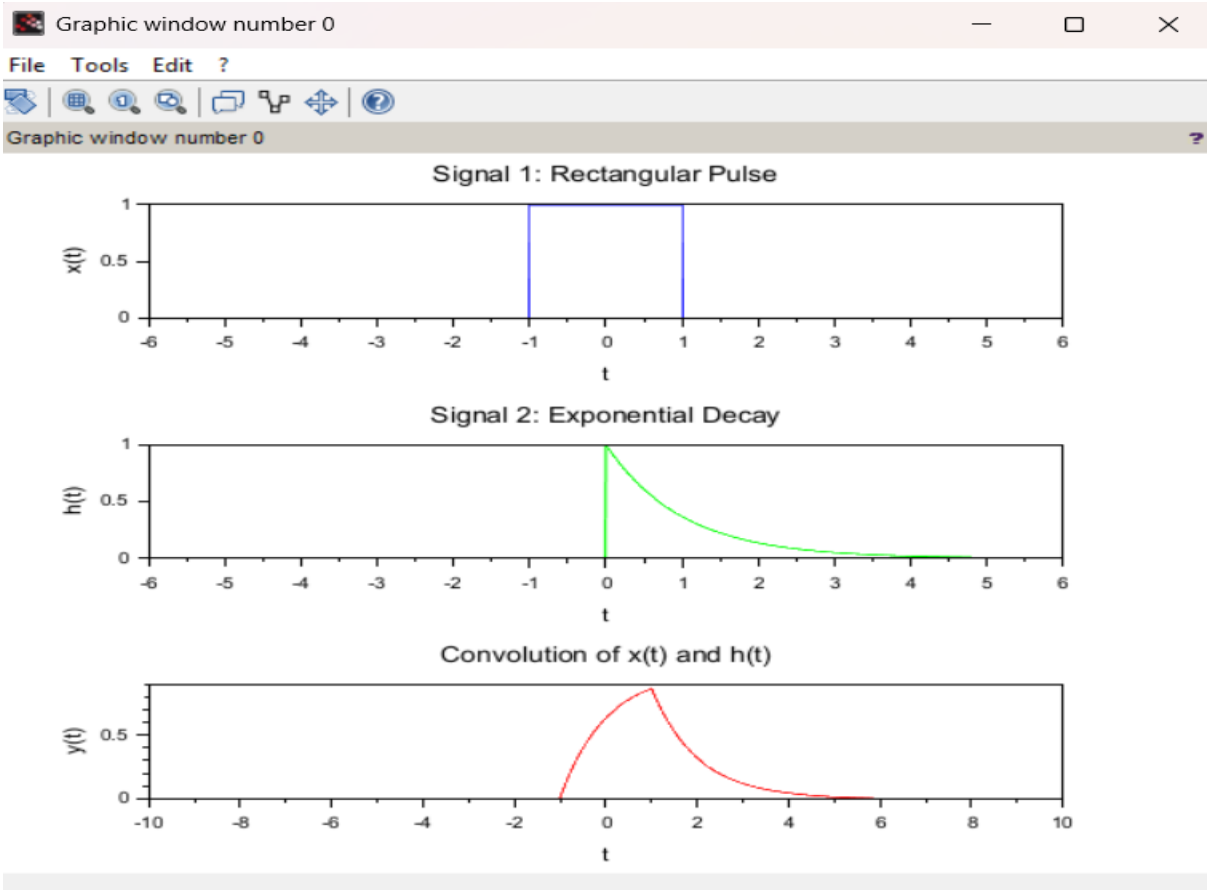
6. CONVOLUTION AND SAMPLING TECHNIQUES

Experiment 6.1

```
// Test Case: Convolution of Continuous-Time Signals


// Define time range and sampling interval
t_min = -5; // Start time
t_max = 5; // End time
dt = 0.01; // Time step
t = t_min:dt:t_max; // Time vector
// Define Signal 1: Rectangular pulse of width 2 centered at t = 0
x_t = (t >= -1) .* (t <= 1);
// Define Signal 2: Exponential decay function
h_t = exp(-t) .* (t >= 0); // Exponential decay, causal
// Perform the convolution
y_t = conv(x_t, h_t) * dt; // Multiply by dt to scale correctly
// Adjust the time vector for the convolution result
t_conv = (2 * t_min):dt:(2 * t_max);
// Plot Signal 1
clf; // Clear the current figure
subplot(3,1,1);
plot(t, x_t, 'b');
xlabel('t');
ylabel('x(t)');
title('Signal 1: Rectangular Pulse');
```

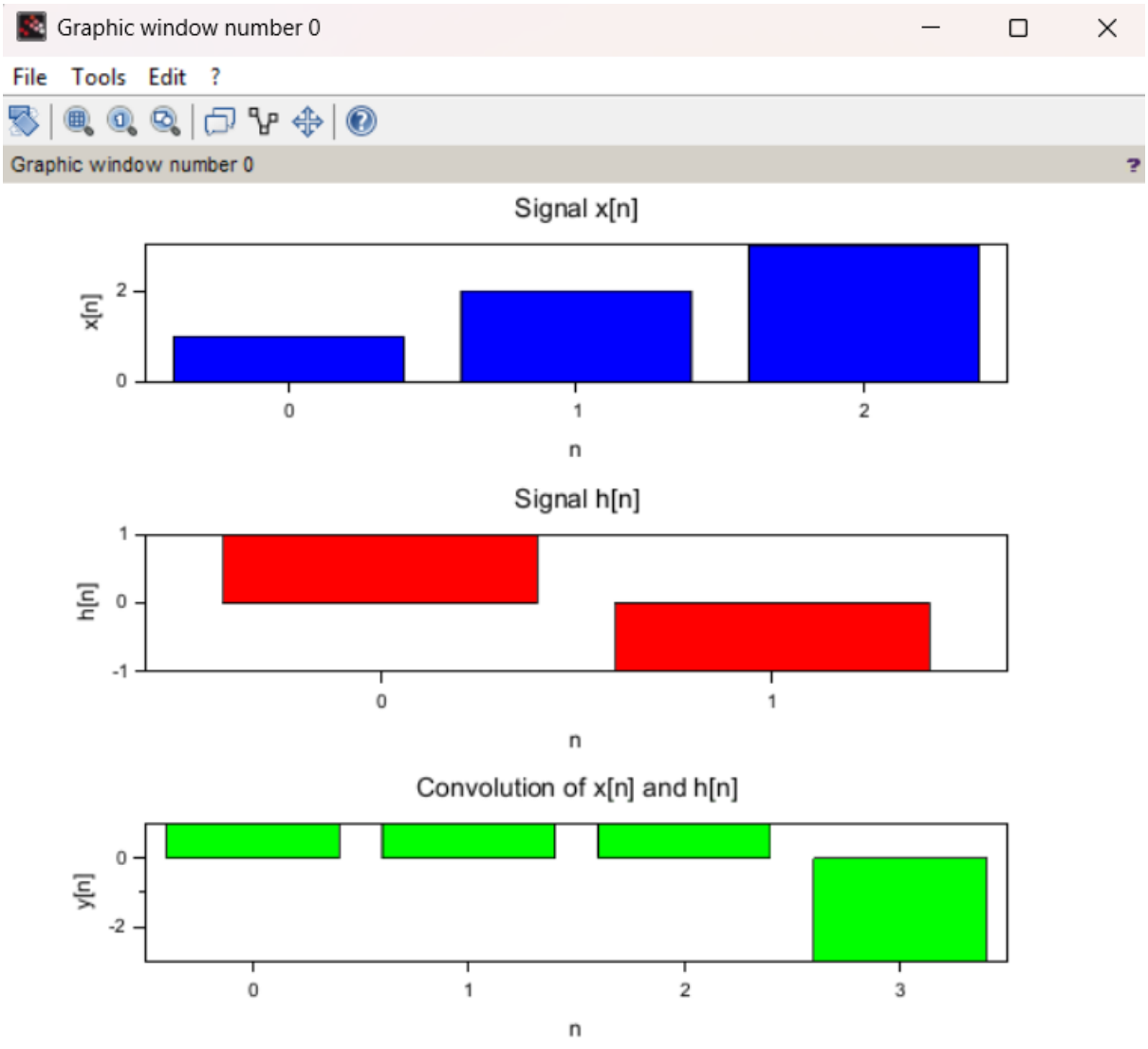
```
// Plot Signal 2
subplot(3,1,2);
plot(t, h_t, 'g');
xlabel('t');
ylabel('h(t)');
title('Signal 2: Exponential Decay');
// Plot the Convolution Result
subplot(3,1,3);
plot(t_conv, y_t, 'r');
xlabel('t');
ylabel('y(t)');
title('Convolution of x(t) and h(t)');
```



Experiment 6.2

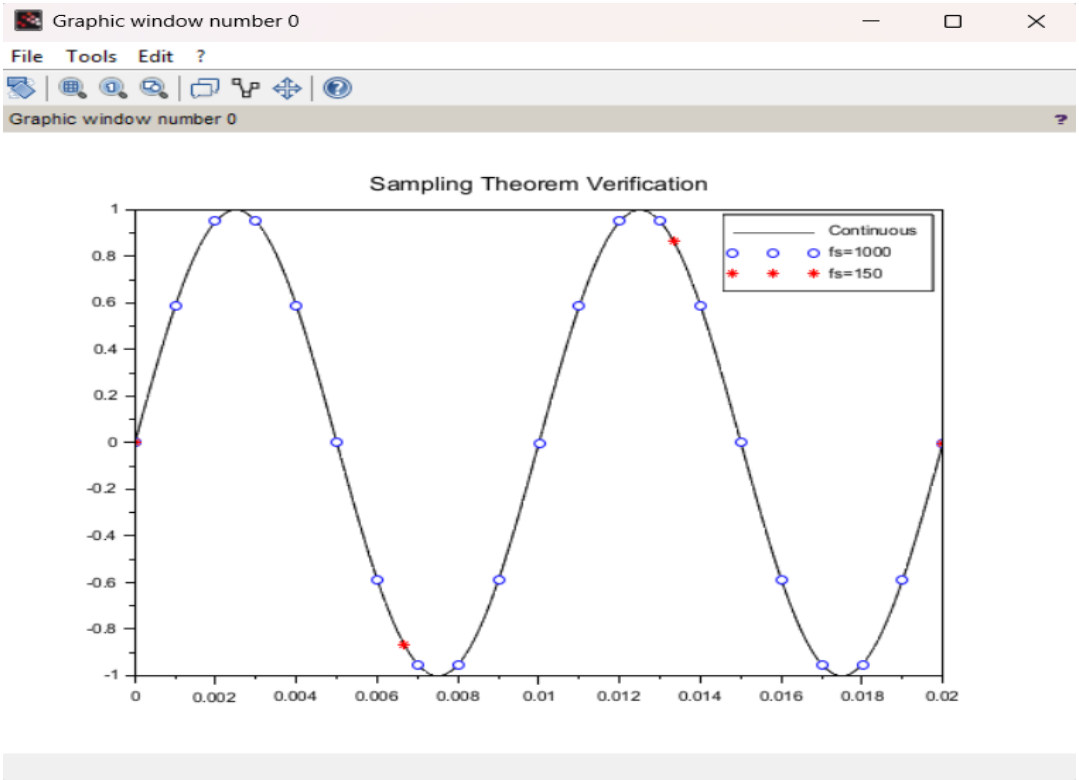
```
// Test Case: Convolution of Discrete-Time Signals
// Define the discrete time signals
x_n = [1, 2, 3]; // Signal x[n]
h_n = [1, -1]; // Signal h[n]
// Compute the convolution
y_n = conv(x_n, h_n); // Convolution of x[n] and h[n]
// Define the time index for the output signal
n_y = 0:(length(x_n) + length(h_n) - 2); // Length of y[n]
// Plot the input signals and their convolution
clf; // Clear the current figure
// Plot x[n]
subplot(3,1,1);
bar(0:length(x_n)-1, x_n, 'b');
xlabel('n');
ylabel('x[n]');
title('Signal x[n]');
// Plot h[n]
subplot(3,1,2);
bar(0:length(h_n)-1, h_n, 'r');
xlabel('n');
ylabel('h[n]');
title('Signal h[n]');
// Plot y[n] (convolution result)
```

```
subplot(3,1,3);
bar(n_y, y_n, 'g');
xlabel('n');
ylabel('y[n]');
title('Convolution of x[n] and h[n]');
```



Experiment 6.3

```
t_cont = 0:1e-4:0.02;
f0 = 100;
x_cont = sin(2*%pi*f0*t_cont);
fs1 = 1000;
n1 = 0:1/fs1:0.02;
x_s1 = sin(2*%pi*f0*n1);
fs2 = 150;
n2 = 0:1/fs2:0.02;
x_s2 = sin(2*%pi*f0*n2);
clf;
plot(t_cont, x_cont, 'k');
plot(n1, x_s1, 'bo');
plot(n2, x_s2, 'r*');
legend("Continuous","fs=1000","fs=150");
```



Experiment 6.4

```
clc;
clear;
clf;

//=====================================
// Aliasing Effect Demonstration
//=====================================

// Continuous-Time Signal
t = 0:0.0001:0.05;
x = sin(2*%pi*300*t) + 0.8*sin(2*%pi*700*t);

//-------------------------------------
// Sampling Parameters
//-------------------------------------
fs = 800;         // Sampling Frequency (Hz)
Ts = 1/fs;

n = 0:Ts:0.05;
xs = sin(2*%pi*300*n) + 0.8*sin(2*%pi*700*n);

//-------------------------------------
```

```
// FFT of Sampled Signal
//-----------------------------------
N = 2048;
Xf = abs(fft(xs, -1));
Xf = [Xf zeros(1, N-length(Xf))];   // Zero Padding
f = (0:N-1)*(fs/N);

//-----------------------------------
// Plotting
//-----------------------------------

// Continuous and Sampled Signal
subplot(2,1,1);

plot(t, x);
plot2d3(n, xs);

legend("Continuous Signal", "Sampled Signal");
xlabel("Time (s)");
ylabel("Amplitude");
title("Continuous vs Sampled Signal");

// Spectrum
subplot(2,1,2);
```

```
plot(f, Xf);


xlabel("Frequency (Hz)");
ylabel("Magnitude");
title("Spectrum of Sampled Signal (Aliasing Effect)");


a = gca();
a.data_bounds = [0 0; fs/2 max(Xf)];
```